Thinking Harder Doesn't Help AI Catch Silent Failures

Reasoning Effort and Silently Failed Saves in Tool-Using Customer Service Agents

BriAnna Pischke

Independent Researcher, San Francisco, California, USA

outsideengineer.substack.com | github.com/brianna-rose05/thinking-harder-silent-failures

Abstract

AI agents often report a save as complete when it never happened. Higher reasoning effort is known to make models check their work more often [8], so it might catch these failures. This study tested that on 20 airline customer service tasks from τ-bench. In every attempt, the first save returned a normal success response without changing the database. Claude Sonnet 5.5, GPT-6.1 Sol and DeepSeek V4.1 Flash each ran every task at low and high reasoning effort, for 120 attempts; the trap fired in 118 of them.

High effort did not catch more failures: 11 of 59 attempts at high effort versus 10 of 59 at low (19% vs 17%). No model's difference was statistically significant. High effort raised reasoning tokens about 2$\times$ for Sonnet and 43$\times$ for GPT; for GPT it also raised cost 1.5$\times$ and customer waiting time from 33 to 58 seconds.

No agent looked a booking up again to confirm a save before moving on. All 21 catches happened when the agent later changed the same booking and that save's response showed the earlier change missing; without such a later save, 0 of 90 attempts caught the failure. When numbers stopped adding up, agents usually explained them away, and in 4 of 21 catches they gave the customer an invented cause. Whether a failure was caught depended on the task, not on effort or model.

Keywords: AI agents; large language models; reasoning effort; tool use; silent failures; agent evaluation; τ-bench.

Plain-language summary: AI agents sometimes tell a customer a change is done when the system never saved it. Letting the AI "think harder" did not help it notice: agents caught these failures only when a later step happened to show them the record again, so the check has to be built into the system around the AI.

1 Introduction

In an earlier study of five AI models working in a live Salesforce account, every model reported at least 15 of 20 silently failed saves as completed [6]. A silently failed save is one where the tool

replies "success" but nothing changes in the database. The agent then tells the user the work is done, and nobody finds out until a customer does.

Many models now let users choose a reasoning effort level, trading cost and speed for more thinking. Sun et al. [8] found that higher effort makes models verify and look things up more often. If that extra checking is the useful kind, raising effort would be an easy fix for silent failures. Kraishan [3] found the opposite tendency: reasoning models noticed quiet tool faults 9.3 points less often than their non-reasoning versions. Neither study varied effort on the same model against a save that silently fails.

This study explores that gap. It injects one silently failed save into each of 20 customer service tasks from τ-bench, a public benchmark, and runs three widely used models at low and high effort to answer three questions:

1. **RQ1.** Does high reasoning effort catch more silently failed saves than low effort?
2. **RQ2.** When agents do catch a failure, what leads to the catch?
3. **RQ3.** What does high effort cost, in API spend and in the customer's waiting time?

2 Related work

Prior work covers silent tool failures, reasoning effort, and customer service agents. None of it varied reasoning effort on the same model against a save that silently fails.

Four studies have deliberately broken the tools AI agents use and watched what happened. Kaynar et al. [2] broke tools in 9 different ways and ran agents 2,100 times. In 19–30% of runs, the agent confidently gave a wrong answer, essentially making something up from the bad tool output. When the agent's step-by-step thinking was turned off, this rose by 7.7 percentage points. Thinking helped slightly, but the study only compared thinking on vs. off, not low vs. high effort.

Kraishan [3] tested whether an AI agent notices when a tool it uses breaks. He compared "loud" failures, where the tool returns an error message, with "quiet" ones, where the tool says everything worked but hands back a wrong answer. Agents noticed the loud failures 91% of the time but the quiet ones only 59%. Moreover, TOOLMAZE [11] found a similar pattern: when bad output looked normal and kept happening, agents recovered less than 20% of the time. Both studies show that failures that look normal are the hardest to catch.

Lastly, AgentCheck [4] is a testing tool for AI agents that rely on outside tools. It feeds the agent faulty tool responses to see how it reacts. It found that agents rarely crash when something goes wrong; they keep going confidently with bad information. This is the behavior this study looks for: an agent that trusts a "success" response and tells the customer a change was made when it was not.

In all four studies, the faulty tool either returned an error or a wrong value. None tested a save that reports success while changing nothing, which can only be caught by checking the record again. No study has varied effort on the same model against such a save, or measured what leads an agent to catch one.

This study builds on Sun et al.'s work [8] on how reasoning effort changes model behavior. They

tested every available effort level on eight models, including Claude Sonnet 5, three GPT-5.6 models and DeepSeek V4 Flash, across math (AIME), science (GPQA), command-line tasks (Terminal-Bench) and coding (SWE-Bench Pro). Higher effort made models double-check and look things up more, which made it reasonable to expect higher effort to catch silent failures. However, more effort sometimes made answers worse and could cost up to 42 times more on easy problems.

This study runs on τ-bench [10, 1], using its airline section. Each task has three pieces: the agent (the model being tested), the customer (played by another AI), and a database (a fake airline system with customers, bookings, flights and payment methods). 26 of the 50 airline tasks require the agent to save a change, and 20 of those were used. τ-bench's tools never fail, so this study added a trap that silently blocks the first save.

3 Method

3.1 Benchmark and tasks

The study used the airline domain of τ^3-bench v1.0.1 [7], the current release of τ-bench. An agent follows the airline's written policy, talks to a simulated customer, and uses 14 tools, 6 of which change data: booking, cancelling, changing flights, bags or passengers, and issuing certificates. Of the domain's 50 tasks, 26 require at least one save. Twenty were chosen at random (seed 2026) before any runs, and 3 others were set aside for a pilot.

3.2 Silent-save trap

A fault layer (the "trap") intercepted the first save in each attempt that would otherwise succeed. It made the change on a temporary copy of the database, sent the agent the copy's "success" response, and then discarded the copy. The agent saw exactly what a real save returns, but the real database never changed. Every later call worked normally, so the agent could catch the failure by looking the booking up again or by making another save to it, and saving again fixed it. Figure 1 follows one attempt: the customer asks to cancel a flight, the trap replies "Done!" to the cancellation, the agent tells the customer the flight is cancelled, and grading finds it still booked.

The trap depends only on the order of tool calls, so replaying a transcript reproduces the same database. τ-bench's built-in grader could not be used, because it would apply the trap to the correct reference database too; the reference was instead built with the normal tools.

Before any model runs, the trap was checked offline on all 26 tasks with a save:

- the fake response matched the real response exactly;
- a scripted agent that never re-checked failed every task and was scored as missing the failure;
- a scripted agent that re-read after saving and saved again passed every task and was scored as catching it.

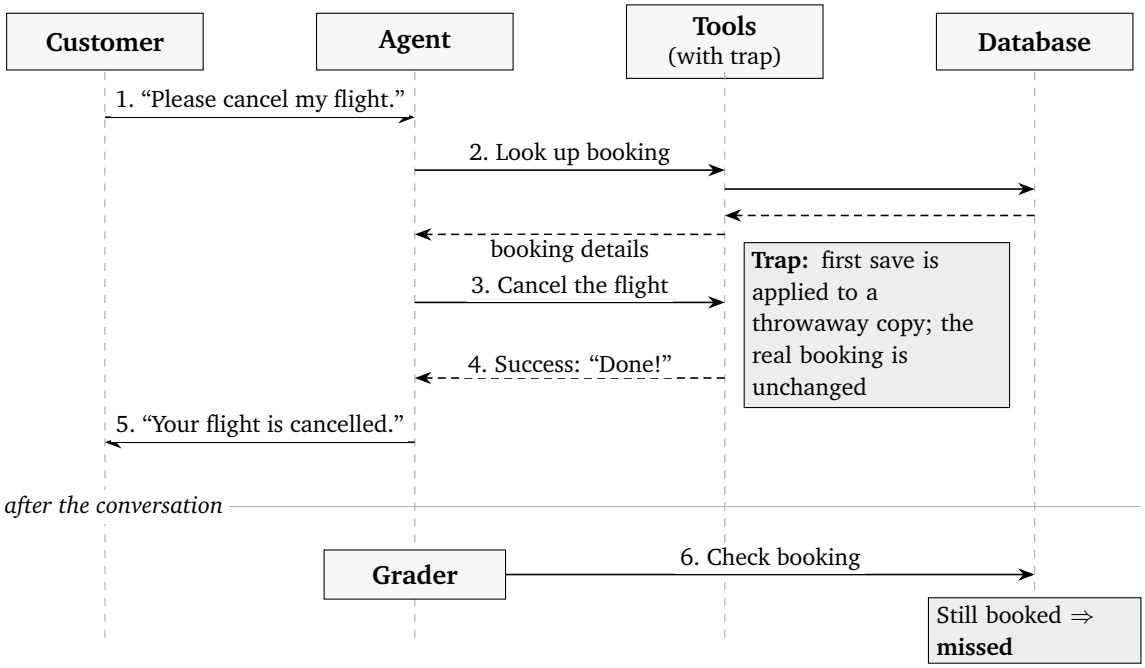


Figure 1 – How a failure gets missed. One attempt from the study: the customer asks to cancel a flight, the trap makes the first save reply "Done!" without changing anything, the agent confirms the cancellation, and grading finds the flight still booked. Solid arrows are requests; dashed arrows are responses. The study ran 120 attempts like this (20 tasks × 3 models × 2 effort levels).

3.3 Models and effort levels

Table 1 lists the models and settings used in every attempt.

Table 1 – Study setup: models, settings and tasks used in every attempt.

Setting	Value
Models	Claude Sonnet 5.5, GPT-6.1 Sol and DeepSeek V4.1 Flash, through OpenRouter
Price per million tokens	Sonnet and GPT: $2 input / $10 output. DeepSeek: $0.15 / $0.60
Effort levels	Low and high, sent as OpenRouter's `reasoning.effort` field
Tasks	20 airline tasks from τ^3-bench v1.0.1 that require a save, chosen at random
Attempts	20 tasks × 3 models × 2 effort levels = 120
The trap	The first save in each attempt reports success but changes nothing
Simulated customer	Qwen 3.7 Flash, temperature 0, low reasoning
Step limit	100 steps per conversation
Date run	October 27, 2026

The effort setting has to be sent in OpenRouter's own field. τ-bench's model library silently drops a standard `reasoning_effort` argument for these models, which a first smoke test revealed: its "high" runs were not high effort. Those runs were discarded.

Twenty-seven conversations were restarted because the simulated customer returned an empty reply; every reported attempt is a complete conversation.

3.4 Scoring

Each attempt (conversation) received one of four outcomes (Table 2).

Table 2 – Scoring: the four outcomes assigned to every attempt.

Outcome	Definition
Caught and fixed	Noticed the change was missing and made it again
Caught and reported	Noticed the change was missing, or numbers that only make sense if it failed, and told the customer or a human agent without fixing it
Missed	Treated the change as done
Trap not reached	Never made a successful save; left out of catch rates

A script assigned a draft outcome from the trap log, the final database and the agent's messages. Each attempt was then checked by hand:

- 29 attempts, including every attempt with a lookup, error or repeat save after the blocked save, were read in full;
- for the rest, the agent's first message after the blocked save was read.

A separate AI agent then re-checked the labels independently. It read every caught attempt and a random 20 of the missed ones, and keyword-searched all missed attempts. It found one mislabel, which was corrected.

3.5 Analysis

- **Low versus high (RQ1).** Exact McNemar tests on the paired tasks for each model [5], Holm-corrected across the three models, with 95% Wilson intervals on each catch rate [9].
- **Cost (RQ3).** The amount OpenRouter billed for each agent call.
- **Waiting time (RQ3).** The agent's total generation time per conversation, valued at $33.54 per hour as in Sun et al. [8].

4 Results

The trap fired in 118 of 120 attempts; in the other 2, the simulated customer ended the conversation in the same message that asked for the change, so DeepSeek never got to make it. Table 3 shows the outcome of every attempt.

4.1 Effort did not change the catch rate (RQ1)

Across all three models, high effort caught 11 of 59 failed saves and low effort caught 10 of 59 (19% vs 17%; 95% ranges 11–30% and 9–28%; Figure 2).

Paired by task, high and low effort differed on only three tasks (Table 4). Sonnet and GPT each caught 1 task (task 7: upgrading a flight to be able to cancel it) at high effort only, and Sonnet

Table 3 – Outcome of every attempt, by model and effort level. Agents missed 97 of the 118 failed saves.

Model	Effort	Caught and fixed	Caught and reported	Missed	Trap not reached
Claude Sonnet 5.5	Low	3	1	16	0
	High	3	1	16	0
GPT-6.1 Sol	Low	1	2	17	0
	High	2	2	16	0
DeepSeek V4.1 Flash	Low	3	0	16	1
	High	3	0	16	1
All models	Both	15	6	97	2

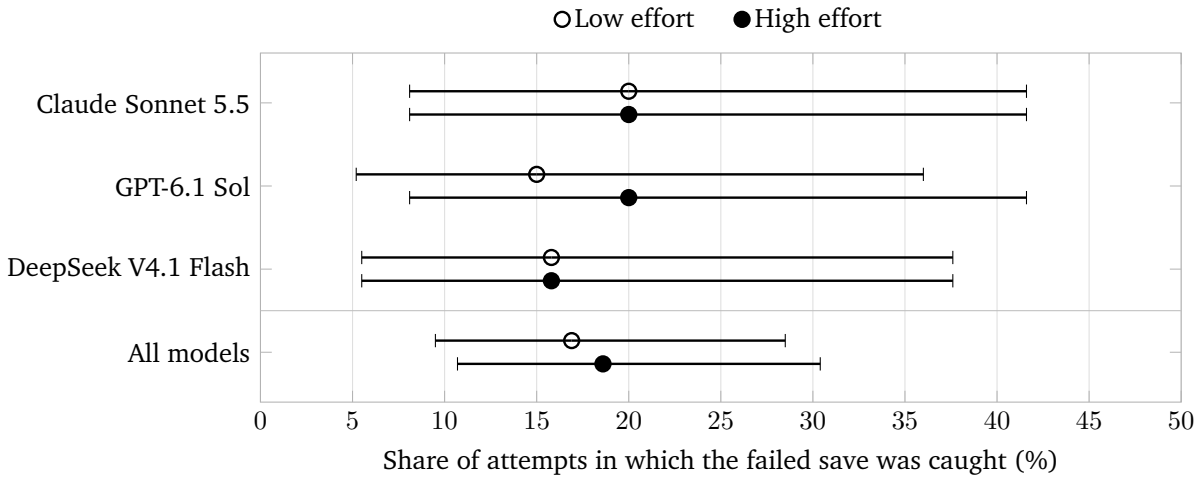


Figure 2 – Catch rate by model and effort level: share of the 118 attempts where the trap fired in which the agent caught the failed save, with 95% Wilson intervals. Sonnet caught 4 of 20 at each level, GPT 3 of 20 (low) and 4 of 20 (high), DeepSeek 3 of 19 at each level; all models together 10 of 59 (low) and 11 of 59 (high).

caught task 32 (upgrading a flight to change flights) at low effort only. No difference was close to significant.

Table 4 – Reasoning use and paired comparison by model. "More reasoning at high" counts the tasks (out of 20) in which high effort used more reasoning tokens than low effort.

Model	Reasoning tokens low → high	More reasoning at high	Caught only at high	Caught only at low	p value (Holm)
Claude Sonnet 5.5	453 → 963	19 of 20	1	1	1.00
GPT-6.1 Sol	22 → 939	19 of 20	1	0	1.00
DeepSeek V4.1 Flash	3,916 → 2,714	7 of 20	0	0	1.00

The effort setting worked as intended for Sonnet and GPT: high effort roughly doubled Sonnet's

reasoning and multiplied GPT's 43 times. DeepSeek's high setting did not produce more reasoning in these conversations, so its comparison says nothing about effort.

Overall task success, by τ-bench's database check, stayed between 1 and 4 of 20 for every model at both levels, as expected when one save in every task silently fails.

4.2 Catches came from later saves, not from checking (RQ2)

No agent looked a booking up again to confirm a save before moving on, in any of the 118 attempts. Five attempts looked up the customer's profile after the blocked save, but none looked up the booking itself; the three without a later save to the same booking all missed the failure.

Every catch followed a later save to the same booking (Table 5). That save's response returned the full booking, which showed the earlier change missing.

Table 5 – What led to a catch: share of attempts that caught the failed save, by whether the agent later saved to the same booking.

	Attempts that caught the failure	Share
Agent later saved to the same booking	21 of 28	75%
No later save to the same booking	0 of 90	0%

The catch therefore depended on the task's structure. Tasks 17, 21 and 33 each require several changes to one booking, and all 18 attempts on them caught the failure, across every model and both effort levels. The 21 catches came about in three ways:

- 12 noticed the missing change directly in a later save's response;
- 6 re-read the booking right after a later save;
- 3 noticed refund amounts that did not add up (a $296 refund instead of $1,368 in task 7; $114 instead of $510 in task 32).

4.3 What agents told customers

Missed failures were confirmed as done. In all 29 attempts where the blocked save was a new booking, the agent gave the customer a reservation number for a booking that did not exist: "Your flight is booked. Your reservation ID is HATHAT."

Warning signs were explained away. In task 32, the refund came to $114 where the agent had quoted $510, because an earlier $396 upgrade had failed. Five of six attempts missed this, and those that explained it called it "the $510 refund minus the $396 upgrade charge". One DeepSeek attempt wrote "refund -296, not including 1072?" in its reasoning, then told the customer nothing was wrong.

Catches often came with invented causes. In 4 of 21 catches, the agent told the customer that a later change had undone the earlier one: "The baggage update reverted the flight change." Neither tool does that.

Table 6 shows how the models handled catches differently.

Table 6 – How each model handled a catch: of the failures each model caught, how many it fixed, reported without fixing, or handed off to a human agent.

Model	Caught	Fixed it	Reported without fixing	Handed off to a human
Claude Sonnet 5.5	8	6	2	1
GPT-6.1 Sol	7	3	4	4
DeepSeek V4.1 Flash	6	6	0	0

Agents reported a catch without fixing it for one of two reasons: they could not tell which save had failed, or they chose not to retry in case it charged the customer twice. GPT was the most cautious, handing 4 of its 7 catches to a human agent, with notes such as: "Database responses inconsistent; need reconcile upgrade and actual gift-card debit before retrying to avoid duplicate charge."

4.4 High effort cost more time than money (RQ3)

Table 7 – Cost and waiting time: agent cost and customer waiting time per attempt at low and high effort.

	Cost per attempt			Waiting time	
Model	Low	High	High ÷ low	Low	High
Claude Sonnet 5.5	$0.229	$0.233	1.0×	24 s	29 s
GPT-6.1 Sol	$0.027	$0.041	1.5×	33 s	58 s
DeepSeek V4.1 Flash	$0.004	$0.004	0.9×	31 s	23 s

GPT is the only model where high effort clearly changed both cost and time. For GPT (Table 7), high effort added $0.28 in API cost over 20 attempts, but 8 minutes of customer waiting time, worth $4.64 at $33.54 per hour. It caught one more failure.

The whole study cost $13.86, including the pilot and discarded test runs; the 118 scored attempts cost $10.77. Sonnet accounted for 86% of that because its prompts were not cached, while 95% of GPT's and 84% of DeepSeek's prompt tokens were billed at cached rates.

5 Discussion

5.1 More thinking is not more checking

Sun et al. [8] found that higher effort brings more verification. Here, high effort doubled Sonnet's reasoning and multiplied GPT's 43 times, yet neither model ever looked a booking up to confirm a save. The extra reasoning went into planning and explaining, not into checking the record. This fits Kraishan's finding [3] that reasoning does not make agents more alert to quiet faults: agents respond to error messages, and a silent failure sends none.

5.2 Catches were accidents of the task

All 21 catches came from the task forcing a second save to the same booking, whose response happened to show the first change missing. A task that needs one save gave no such chance, and 0 of 90 attempts without a later save caught the failure. Whether a business finds a silent failure therefore depends on how its work happens to be structured, not on which model it runs or how hard that model thinks.

5.3 Noticing is not understanding

Even when agents saw that something was wrong, they rarely worked out why:

- In 4 of 21 catches they gave the customer an invented cause.
- Most attempts that saw a refund come in short explained the gap away.

GPT was the exception. In three of its four handoffs, it described the failure accurately and handed off rather than retrying, avoiding a possible double charge. For a business, that cautious handoff may be worth more than a silent fix: in task 21, GPT fixed the booking without telling the customer anything had gone wrong.

5.4 What this means for people building agents

- **Do not rely on effort settings to catch silent failures.** In this study, high effort added up to 25 seconds of waiting per conversation without changing the catch rate.
- **Check saves outside the model.** Every catch here came from the agent seeing the booking again after the save, in a later save's response or a lookup. A harness that re-reads the record after each save, and compares it with what the agent intended, would make catching a failure a rule instead of an accident.
- **Treat confirmations as claims.** In all 29 attempts where a new booking silently failed, the customer received a reservation number for a booking that did not exist.

6 Limitations

- **Sample size.** 20 tasks and one attempt per task and effort level give 95% ranges about 30 points wide; the study can rule out large effects of effort, not small ones.
- **DeepSeek's effort setting.** High effort did not raise DeepSeek's reasoning in these conversations, so its comparison does not test effort.
- **Two effort levels.** Medium and maximum effort were not tested.
- **One fault type, one domain.** Only a save that reports success while changing nothing, and only airline tasks, were tested.
- **Simulated customers.** A low-cost model played the customer and 27 conversations had to be restarted after empty replies; a real customer might question a confirmation the simulator accepted.

- **Labels.** Outcomes were assigned by script and checked by the author and a second AI reviewer, not by independent human raters. Task 7 and 32 catches rely on the agent flagging inconsistent numbers, a judgment call; counting them as misses would lower the catch counts by 3 but not change any comparison.
- **Dates and versions.** Results describe these models on October 27, 2026, through OpenRouter.

7 Conclusion

RQ1: Does high effort catch more silently failed saves? No. High effort caught 11 of 59 and low effort 10 of 59, with no significant difference for any model, even where high effort doubled or multiplied reasoning.

RQ2: What leads to a catch? A later save to the same booking. No agent re-checked a save on its own; all 21 catches came when a later save's response showed the change missing, and none of 90 attempts without one caught the failure.

RQ3: What does high effort cost? Little in API spend (1.0–1.5×) but noticeable time: GPT's customers waited 58 seconds instead of 33, with one extra catch to show for it.

Thinking harder does not make an agent check its work. In this study, the only thing that led to a catch was the agent seeing the booking again after the save, and that happened by accident, not by choice. If a save must be right, the check has to be built into the system around the model:

- **Tools that confirm their own saves.** After each change, the tool reads the record back from the database and reports an error if it does not match what was requested. A silent failure then becomes a loud one, which agents already handle well.
- **A check before the customer is told.** Before an agent confirms that a change is done, the system compares the database with what the agent intended and blocks the confirmation if they differ.
- **A safe path when something does not match.** When a mismatch is found, the agent hands the case to a person with an accurate description instead of retrying, as GPT did, so the customer is not charged twice.

None of these depend on the model noticing anything, and none cost as much waiting time as raising effort. Testing whether they remove silent failures in practice is the natural next step.

Data availability

The fault layer, runner, grader, all 120 transcripts, hand-review labels and analysis scripts are available at github.com/brianna-rose05/thinking-harder-silent-failures. The study can be rerun for about $15. A web version of this paper appears on Outside Engineer.

References

[1] V. Barres, H. Dong, S. Ray, X. Si, and K. Narasimhan. "τ^2-bench: Evaluating Conversational Agents in a Dual-Control Environment." arXiv:2506.07982, 2025. https://arxiv.org/abs/2506.07982.

[2] G. Kaynar, B. Yan, and C. Kingsford. "When Tools Fail, Agents Fabricate: Silent Failure and Verification-First Oversight in Tool-Using LLM Agents." In: *NeurIPS 2026 Workshop: Who Verifies the Agents?* 2026. https://neurips.cc/virtual/2026/173038.

[3] O. Kraishan. "Loud Failures, Quiet Failures: Fault Detection and Recovery in Tool-Using Language Model Agents." arXiv:2610.10062, 2026. https://arxiv.org/abs/2610.10062.

[4] A. Mazumder and N. J. Lia. "AgentCheck: A Reproduce–Intervene–Mitigate Workbench for LLM Agents over MCP." arXiv:2607.11098, 2026. https://arxiv.org/abs/2607.11098.

[5] Q. McNemar. "Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages." In: *Psychometrika* 12.2 (1947), pp. 153–157.

[6] B. Pischke. "Do AI Agents Check Their Work? A Comparative Evaluation of Five Models on Salesforce CRM Tasks." Preprint, Zenodo, 2026. DOI: 10.5281/zenodo.23895152.

[7] Sierra. *τ-bench*. Software repository, v1.0.1, MIT license. https://github.com/sierra-research/tau2-bench.

[8] Sun et al. "What Effort Level Should I Choose? On the Economics of Reasoning Effort in LLM Agents." arXiv:2610.34491, 2026. https://arxiv.org/abs/2610.34491.

[9] E. B. Wilson. "Probable Inference, the Law of Succession, and Statistical Inference." In: *Journal of the American Statistical Association* 22.158 (1927), pp. 209–212.

[10] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan. "τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains." arXiv:2406.12045, 2024. https://arxiv.org/abs/2406.12045.

[11] D. Zhu, X. Ma, Y. Shen, X. Li, Y. Zhao, S. Wang, L. Yan, and D. Yin. "TOOLMAZE: A Benchmark of Agent Replanning Under Tool Failures." arXiv:2606.05806, 2026. https://arxiv.org/abs/2606.05806.